
Reactjs

<https://react.dev/>

— Julián Pizarro Pertúz —
Universidad del Magdalena

Que es?

React es una librería de JavaScript declarativa, eficiente y flexible para construir interfaces de usuario. Permite componer IUs complejas de pequeñas y aisladas piezas de código llamadas “componentes”. El objetivo de facilitar el desarrollo de **aplicaciones en una sola página (Single Web Application)**

```
class HelloMessage extends React.Component {
  render() {
    return (
      <div>
        Hello {this.props.name}
      </div>
    );
  }
}

ReactDOM.render(
  <HelloMessage name="Taylor" />,
  document.getElementById('hello-example')
);
```

Crear React App

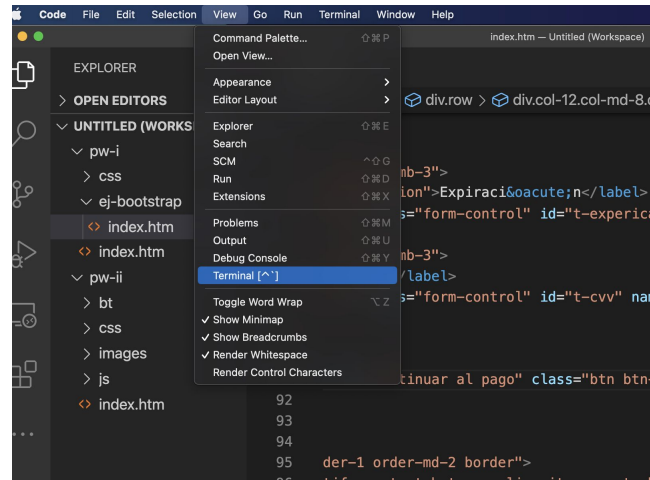
Prerrequisitos

NodeJS -> <https://nodejs.org/en/download/>

VSCode -> <https://code.visualstudio.com/Download>

Generando la App

- Desde la consola ejecutar el comando **npm install -g create-react-app**
- En VSCode seleccionar menú View -> Terminal
 - Ejecutar el comando **npm install -g create-react-app**
 - Ejecutar comando **create-react-app nombredelaapp**



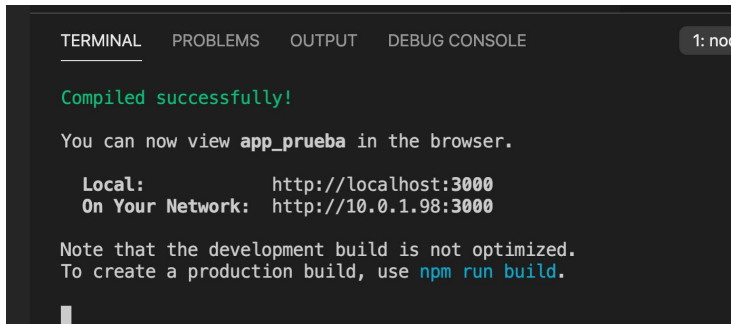
```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  1: node
MacBook-Pro-de-JEFE-LABSIIS:pw-ii jefe-labsis$ create-react-app app_prueba

Creating a new React app in /Users/jefe-labsis/pw-ii/app_prueba.

Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts with cra-template...
```

Crear React App

- Abrir el proyecto en VSCode
 - File -> Open
- Iniciar la aplicación
 - Ejecutar desde la terminal **npm start**
- Navegar en la aplicación
 - `http://localhost:3000/`



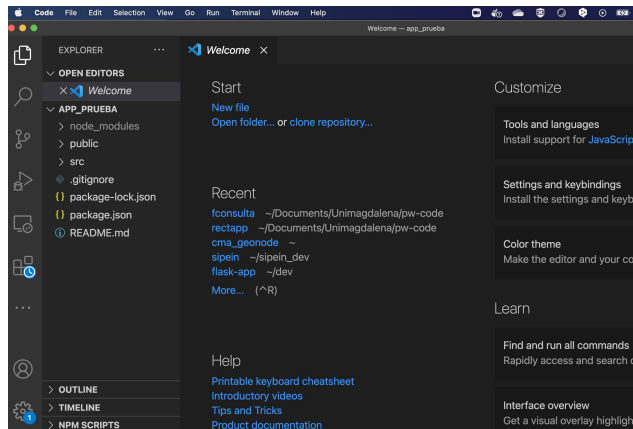
```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  1: nod

Compiled successfully!

You can now view app_prueba in the browser.

Local:      http://localhost:3000
On Your Network:  http://10.0.1.98:3000

Note that the development build is not optimized.
To create a production build, use npm run build.
```



Estructura de una app

node_modules/ contiene todas las bibliotecas de JavaScript externas que utiliza la aplicación. Rara vez necesitará abrirlo.

public/ contiene algunos archivos de imagen, HTML y JSON. Estos son los directorios root de su proyecto

src/ contiene el código JavaScript React de su proyecto. La mayor parte del trabajo que haga se encontrará en ese directorio.

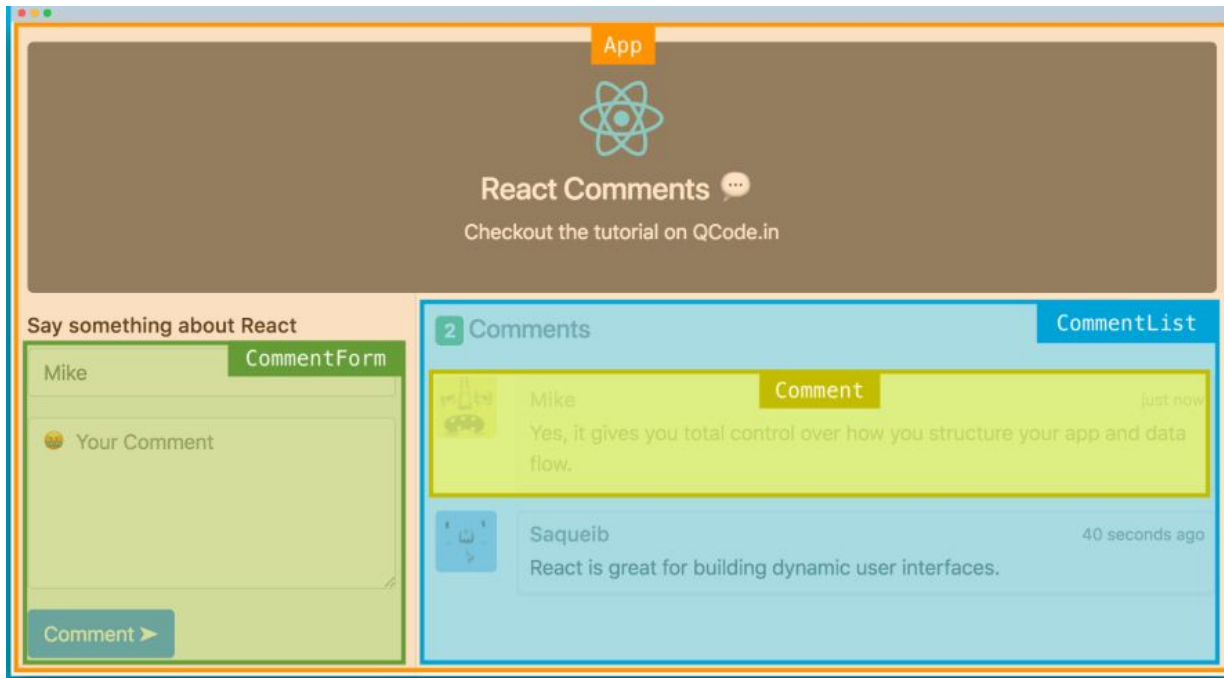
.gitignore contiene algunos directorios y archivos predeterminados que git, su control de código fuente, ignorará, como el directorio node_modules.

README.md es un archivo markdown que contiene mucha información útil sobre Create React App

package.json. Contiene metadatos sobre su proyecto, como el título, el número de versión y las dependencias.

Componentes

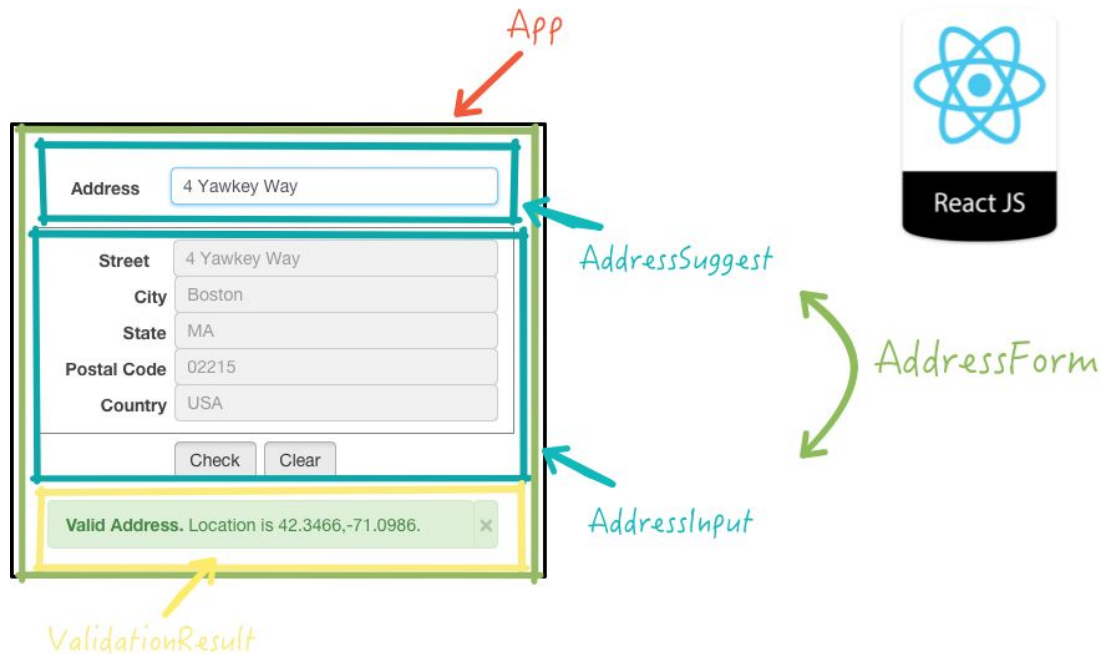
Los componentes permiten separar la interfaz de usuario en piezas independientes, reutilizables y pensar en cada pieza de forma aislada



<https://ashwingoel.com/blog/a-comprehensive-guide-to-creating-your-first-react-app/>

Componentes

Los componentes permiten separar la interfaz de usuario en piezas independientes, reutilizables y pensar en cada pieza de forma aislada



Tipos de componentes

Componentes funcionales (Stateless): Estos componentes son puramente de presentación (sin funcionalidad, sólo elementos de salida de la interfaz de usuario) y pueden ser simplemente representados por una función.

```
const Greeting = () => <h1>Hi, I'm a dumb component!</h1>;
```

```
const Footer = props => {  
  return (  
    <footer>  
      <p>Copyright ©</p>  
    </footer>  
  );  
};
```

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```


Tipos de componentes

Componentes de clase (Stateful): Estos componentes se implementan utilizando una clase y tienen la capacidad de aplicar lógicas, mantener el estado y renderizar el componente.

```
class Greeting extends React.Component {  
  render() {  
    return <h1>Hi, I'm a smart component!</h1>;  
  }  
}
```

```
class Header extends React.Component {  
  render() {  
    return (  
      <header>  
        <div>Logo</div>  
        <nav>  
          <a href="#">Home</a>  
          <a href="#">Blog</a>  
          <a href="#">Photos</a>  
          <a href="#">Contact</a>  
        </nav>  
      </header>  
    );  
  }  
}
```

JSX

JSX es una sintaxis similar a XML/HTML. Añade la sintaxis XML/HTML a Javascript.

JSX es una sintaxis para los componentes de React que nos permite escribir HTML dentro de JS.

Dado que JSX es más cercano a JavaScript que a HTML, React DOM usa la convención de nomenclatura camelCase en vez de nombres de atributos HTML. Por ejemplo, **class** se vuelve **className** en JSX, y **tabindex** se vuelve **tabIndex**

Usa llaves(**{ }**) para insertar **expresiones de JavaScript** en nuestro código JSX

```
const element = (  
  <div>  
    <h1>Hello!</h1>  
    <h2>Good to see you here.</h2>  
  </div>  
);
```

```
const element = <img src={user.avatarUrl} />;
```

Propiedades de los componentes (Props)

props: son una forma de hacer que los componentes sean dinámicos pasando propiedades/datos de un componente a otro. **Son inmutables**, es decir, **no se pueden modificar**. Se trata simplemente de **pasar** alguna **información** del **componente padre** al **componente hijo**.

```
import React from "react";
import PropTypes from "prop-types";

const PostItem = props => {
  return (
    <div>
      <h1>{props.title}</h1>
      <p>{props.content}</p>
    </div>
  );
};

PostItem.propTypes = {
  title: PropTypes.string.isRequired,
  content: PropTypes.string.isRequired
};

export default PostItem;
```

```
class PostList extends React.Component {
  render() {
    return (
      <div>
        <PostItem
          title="Post Title 1"
          content="Lorem ipsum dolor sit amet"
        />
        <PostItem
          title="Post Title 2"
          content="Lorem ipsum dolor sit amet, consectetur."
        />
      </div>
    );
  }
}
```

Ejemplo

APP

Gestión de estudiantes

Crear/editar/borrar estudiantes

Header

Estudiantes

StudentList



Luisa Norte

Ingeniería de sistemas

3 semestre

Ver detalle

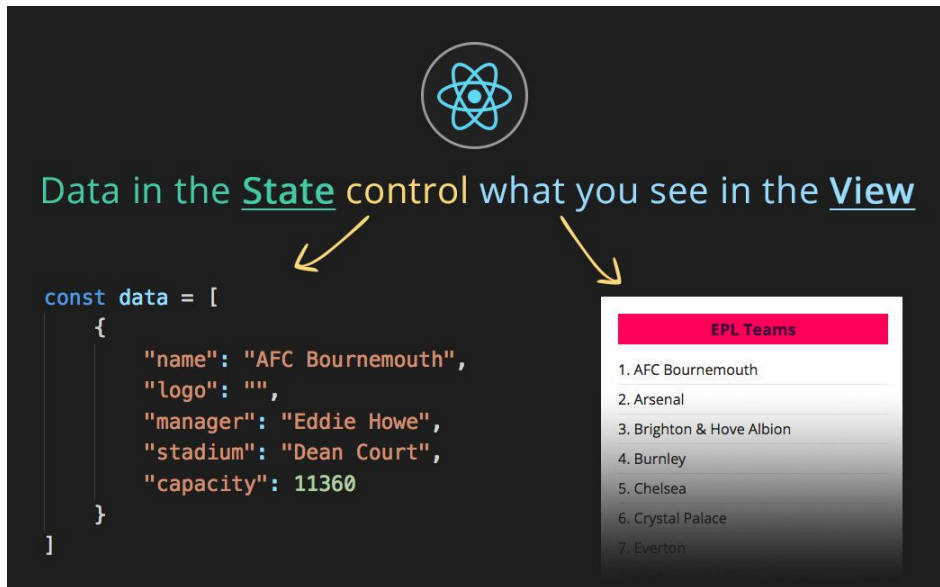
StudentItem

Copyright ©

Footer

Estados

El estado es una variable en cada clase de componente que tiene la capacidad de almacenar información dinámicamente. Si el estado cambia, el componente y sus hijos se volverán a renderizar automáticamente..



The diagram illustrates the relationship between State and View in React. At the top, the React logo is shown. Below it, the text "Data in the State control what you see in the View" is displayed. Two yellow arrows point from the words "State" and "View" to a code snippet and a UI component, respectively.

```
const data = [
  {
    "name": "AFC Bournemouth",
    "logo": "",
    "manager": "Eddie Howe",
    "stadium": "Dean Court",
    "capacity": 11360
  }
]
```

The UI component, titled "EPL Teams", displays a list of teams:

- 1. AFC Bournemouth
- 2. Arsenal
- 3. Brighton & Hove Albion
- 4. Burnley
- 5. Chelsea
- 6. Crystal Palace
- 7. Everton

Estados

- Hay varias maneras de inicializar el estado

Desde el constructor

```
constructor(props) {  
  super(props);  
  
  this.state = {  
    posts: [  
      { id: 1,  
        title: "Post Title 1",  
        content: "Lorem ipsum dolor"  
      },  
      {id: 2,  
        title: "Post Title 2",  
        content: "Lorem ipsum dolor sit a"  
      }  
    ]  
  };  
}
```

Como propiedad de la clase

```
state = {  
  posts: [  
    {  
      id: 1,  
      title: "Post Title 1",  
      content:  
        "Lorem ipsum dolor sit  
amet"  
    },  
    {  
      id: 2,  
      title: "Post Title 2",  
      content:  
        "Lorem ipsum "  
    }  
  ]  
};
```

Estados

Para acceder al estado de un componente lo haremos a través de **this.state**.

```
render() {  
  let posts = this.state.posts;  
  return (  
    <div>  
      {posts.map(post => (  
        <PostItem key={post.id} post={post} />  
      ))}  
    </div>  
  );  
}
```

Estados

Para modificar propiedades del estado, no se hace con una asignación del valor, si no que se hace utilizando la función **this.setState({})**.

```
class PostList extends React.Component {  
  ...  
  handleSubmit = event => {  
    event.preventDefault();  
    const lastPost = this.state.posts.slice(-1).pop();  
    let newPost = {  
      id: lastPost.id + 1,  
      title: event.target.title.value,  
      content: event.target.content.value  
    };  
    console.log(newPost);  
  
    const posts = [...this.state.posts, newPost];  
    this.setState({ posts: posts });  
  };  
  render() {  
    let posts = this.state.posts;  
    return (  
      <div>  
        <PostForm onSubmit={this.handleSubmit} />  
        ...  
      </div>  
    );  
  }  
}
```


Manejo de eventos

- Los eventos de React se nombran usando camelCase, en vez de minúsculas.

`onClick` en vez de `onclick`

- Con JSX pasas una función como el manejador del evento, en vez de un string.

```
<button onClick={activateLasers}>  
  Activate Lasers  
</button>
```

- React no puedes retornar false para prevenir el comportamiento por defecto. Debes, explícitamente, llamar **preventDefault**

```
function ActionLink() {  
  function handleClick(ev) {  
    ev.preventDefault();  
    console.log('The link was clicked.');  }  
  return (  
    <a href="#" onClick={handleClick}>Click me</a>  
  );  
}
```

Manejo de eventos

- Los manejadores de eventos para los componentes pueden ser creados como métodos de clase de ES6. Para este caso es **OBLIGATORIO** ligar (**bind**) el evento al objeto que lo originó para el correcto funcionamiento de **this**

```
class Toggle extends React.Component {
  constructor(props) {
    super(props);
    this.state = {isToggleOn: true};
    // Este enlace es necesario para hacer que `this` funcione en el callback
    this.handleClick = this.handleClick.bind(this);
  }
  handleClick() {
    this.setState(state => ({
      isToggleOn: !state.isToggleOn
    }));
  }
  render() {
    return (
      <button onClick={this.handleClick}>
        {this.state.isToggleOn ? 'ON' : 'OFF'}
      </button>
    );
  }
}
```

Manejo de eventos

- Para evitar tener que usar el método **bind()** para solucionar el problema de **this**, se recomienda el uso de funciones flecha. Con las funciones de flecha, **this** siempre representará el objeto que definió la función de flecha.

```
class LoggingButton extends React.Component {  
  // Esta sintaxis nos asegura que `this` está ligado dentro de handleClick  
  handleClick = () => {  
    console.log('this is:', this);  
  }  
  render() {  
    return (  
      <button onClick={this.handleClick}>  
        Click me  
      </button>  
    );  
  }  
}
```

Manejo de eventos

Para pasar parámetros al manejador de eventos existen dos posibilidades.

- Usando funciones flecha

```
class Football extends React.Component {  
  shoot = (a) => {  
    alert(a);  
  }  
  render() {  
    return (  
      <button onClick={ (ev) => this.shoot("Goal") }>Take the shot!</button>  
    );  
  }  
}
```

Manejo de eventos

Para pasar parámetros al manejador de eventos existen dos posibilidades.

- haciendo una ligadura al objeto **this** con **bind()**. El PRIMER argumento debe ser **this**

```
class Football extends React.Component {  
  shoot(a) {  
    alert(a);  
  }  
  render() {  
    return (  
      <button onClick={this.shoot.bind(this, "Goal")}>Take the shot!</button>  
    );  
  }  
}
```

Manejo de eventos - Objeto de evento de React

Los manejadores de eventos tienen acceso al evento de React que desencadenó la acción.

- mediante funciones flecha. El objeto event debe pasarse como parámetro

```
class Football extends React.Component {  
  shoot = (a, event) => {  
    console.log(b.type);  
    /*  
     'event' represents the React event that triggered the function,  
     in this case the 'click' event  
    */  
  }  
  render() {  
    return (  
      <button onClick={(ev) => this.shoot("Goal", ev)}>Take the shot!</button>  
    );  
  }  
}
```

Manejo de eventos - Objeto de evento de React

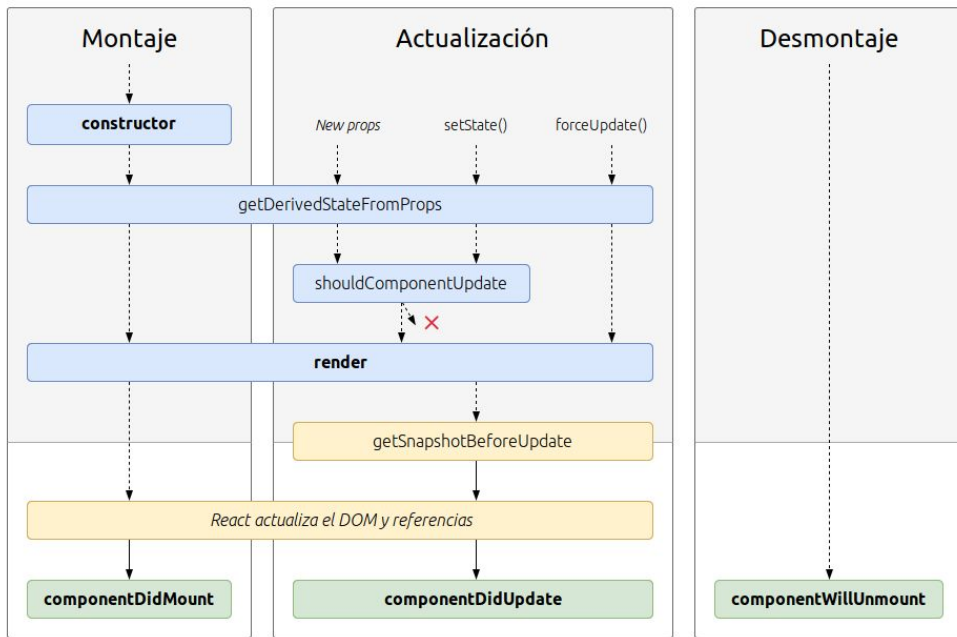
Los manejadores de eventos tienen acceso al evento de React que desencadenó la acción.

- haciendo una ligadura al objeto **this** con **bind()**. El objeto event se envía como el último parámetro

```
class Football extends React.Component {  
  shoot = (a, event) => {  
    alert(event.type);  
    /*  
     'event' represents the React event that triggered the function,  
     in this case the 'click' event  
    */  
  }  
  render() {  
    return (  
      <button onClick={this.shoot.bind(this, "Goal")}>Take the shot!</button>  
    );  
  }  
}
```

Ejemplo

Ciclo de vida de un Componente

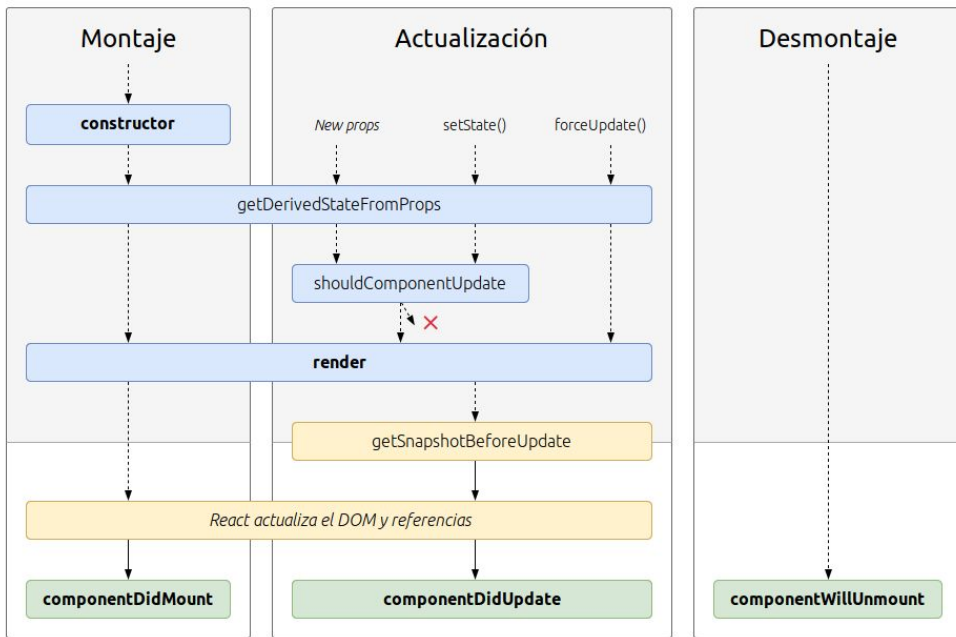


See project on [GitHub](#)

- **constructor(props)**: método de JavaScript que se ejecuta antes de que se monte el componente. Lo podemos utilizar para inicializar el estado del componente con `this.setState` o bindear eventos.
- **render()**: es el único método requerido en los class components y contiene los componentes que se van a renderizar.
- **componentDidMount()**: se ejecuta inmediatamente después de que el componente se haya montado. Por ejemplo, lo que podríamos hacer en este método es una llamada a una API.
- **componentDidUpdate(prevProps, prevState, snapshot)**: se ejecuta inmediatamente después de que ocurra una actualización en el componente.
- **componentWillUnmount()**: se ejecuta inmediatamente después de que un componente se haya desmontado y eliminado.

Ciclo de vida de un Componente

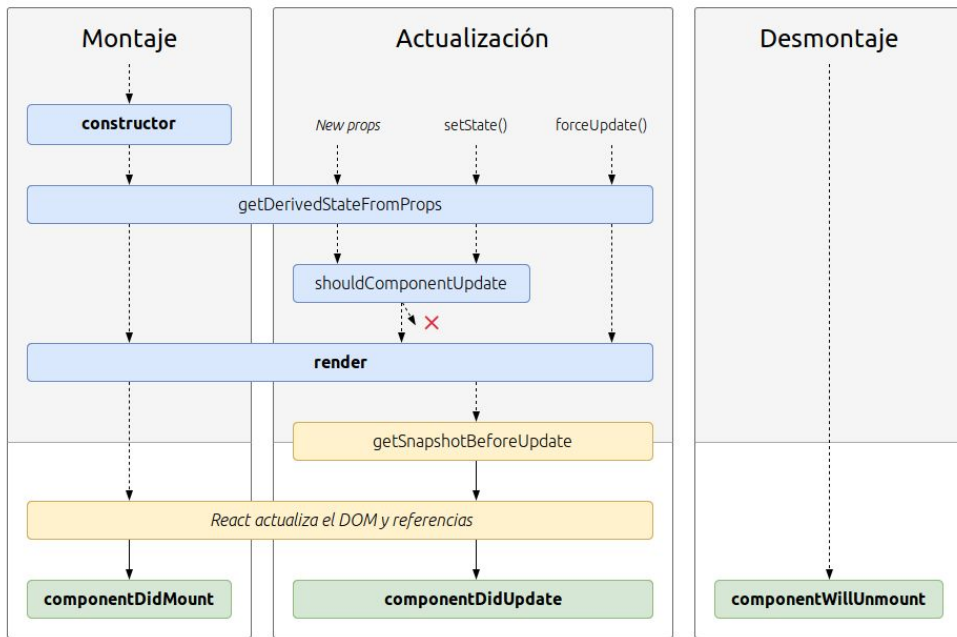
menos común:



See project on [GitHub](#)

- **shouldComponentUpdate(nextProps, nextState):** se ejecuta cuando se hayan actualizado las propiedades o estado y nos permite indicar con true o false, si el componente se tiene que re-renderizar o no.
- **static getDerivedStateFromProps(props, state):** se ejecuta cada vez que se renderiza un componente. Se devuelve un objeto si se quiere que el estado se actualice y null si no se quiere. Se utiliza cuando el estado depende de las propiedades.
- **getSnapshotBeforeUpdate(prevProps, prevState):** se ejecuta inmediatamente antes de que el DOM se actualice y nos permite acceder a cierta información del DOM antes de que se hagan los cambios. El valor que se devuelva será pasado a través del parámetro snapshot a **componentDidUpdate**.

Ciclo de vida de un Componente



See project on [GitHub](#)

- Los siguientes métodos, introducidos en React 16, nos permitirán controlar los errores lanzados desde componentes hijos:
- **getDerivedStateFromError(error)**: recibe el error por parámetro y se devuelve un objeto para actualizar el estado, si no se quiere actualizar el estado se devuelve null. Así, si devolvemos un objeto, podremos acceder al error o a lo que devolvamos en el objeto desde `this.state`.
- **componentDidCatch(error, info)**: se ejecuta después de que un componente hijo haya lanzado un error.

RUTAS con REACT ROUTER

React Router que nos permite añadir rutas (navegación) a cualquier proyecto React. Podemos:

- Establecer rutas en nuestra aplicación (Ejem. Home, Inicio de sesión, etc.)
- Crear páginas públicas y privadas.
- Realizar redirecciones a otras páginas según ciertas condiciones
- Acceder al historial de navegación
- Crear rutas para páginas 404 (no encontradas)

Para utilizar la librería es necesario

npm install react-router-dom --save

RUTAS con REACT ROUTER

Desde el componente principal (App.js) se debe importar { **BrowserRouter as Router, Route, Link**}

```
import {BrowserRouter as Router, Route, Link} from "react-router-dom";
```

- **BrowserRouter (Router):** Este inyecta propiedades a nuestro componente para poder acceder al historial de navegación, realizar redirecciones, etc.
- **Route:** Es el componente que utilizamos para crear nuestras rutas a otras páginas (componentes)
- **Link:** Este es utilizado para los enlaces (igual como etiquetas <a>)

RUTAS con REACT ROUTER

Desde el componente principal (App.js) se debe importar { **BrowserRouter as Router, Route, Link**}

```
import {BrowserRouter as Router, Route, Link} from "react-router-dom";
```

- **BrowserRouter (Router):** Este inyecta propiedades a nuestro componente para poder acceder al historial de navegación, realizar redirecciones, etc.
- **Route:** Es el componente que utilizamos para crear nuestras rutas a otras páginas (componentes)
- **Link:** Este es utilizado para los enlaces (igual como etiquetas <a>)

RUTAS con REACT ROUTER

Route recibe las siguientes propiedades:

path: la ruta en la que se renderizará el componente en forma de cadena de texto

exact: un booleano para definir si queremos que la ruta tiene o no que ser exacta para renderizar un componente. Eg: `/index !== /index/all`.

strict: un booleano para definir si queremos que el último slash sea tomado en cuenta para renderizar un componente. Eg: `/index !== /index/`.

sensitive: un booleano para definir si queremos distinguir entre minúsculas y mayúsculas, y tomar esto en cuenta para renderizar un componente. Eg: `/index !== /Index`

component: recibe un componente a renderizar. Crea un nuevo elemento de React cada vez. Esto causa que el componente se monte y desmonte cada vez (no actualiza).

render: recibe un método que retorna un componente. A diferencia de `component` no remonta el componente.

children: funciona muy similar a `render`, pero con algunas adiciones. Hoy no voy a ahondar mucho en esta propiedad. Con los 2 métodos de renderizando anteriores será suficiente para lo que queremos hacer.

```
import { BrowserRouter as Router, Route, Switch } from
"react-router-dom";

...

import Home from "../pages/Home";
import About from "../pages/About";
import Contact from "../pages/Contact";
...
class App extends Component {
  render() {
    return (
      <Router>
        <div>
          <Header />

          <Switch>
            <Route exact path="/" component={Home} />
            <Route path="/about" component={About} />
            <Route path="/contact" component={Contact} />
          </Switch>

          <Footer />
        </div>
      </Router>
    );
  }
}
```

RUTAS con REACT ROUTER

Link recibe las siguientes propiedades:

to: una locación en forma de cadena de texto. Esta será la locación a la que navegaremos cuando le damos click a un hipervínculos.

replace: un booleano que nos permite modificar el comportamiento de Link. Cuando este es verdadero en lugar de agregar una nueva locación a la historia, este reemplaza la locación actual.

```
import React from "react";
import { Link } from "react-router-dom";

// class component
class Header extends React.Component {
  render() {
    return (
      <header>
        <div>Logo</div>
        <nav className="nav">
          <Link to="/">Home</Link>
          <Link to="/about">About</Link>
          <Link to="/contact">Contact</Link>
        </nav>
      </header>
    );
  }
}

export default Header;
```


Ejercicio

1. Crear aplicación

`npx create-react-app ejercicio1`

2. Crear estructura del proyecto

+components

+Header

+Product

+GridProduct

+ProductItem

+pages

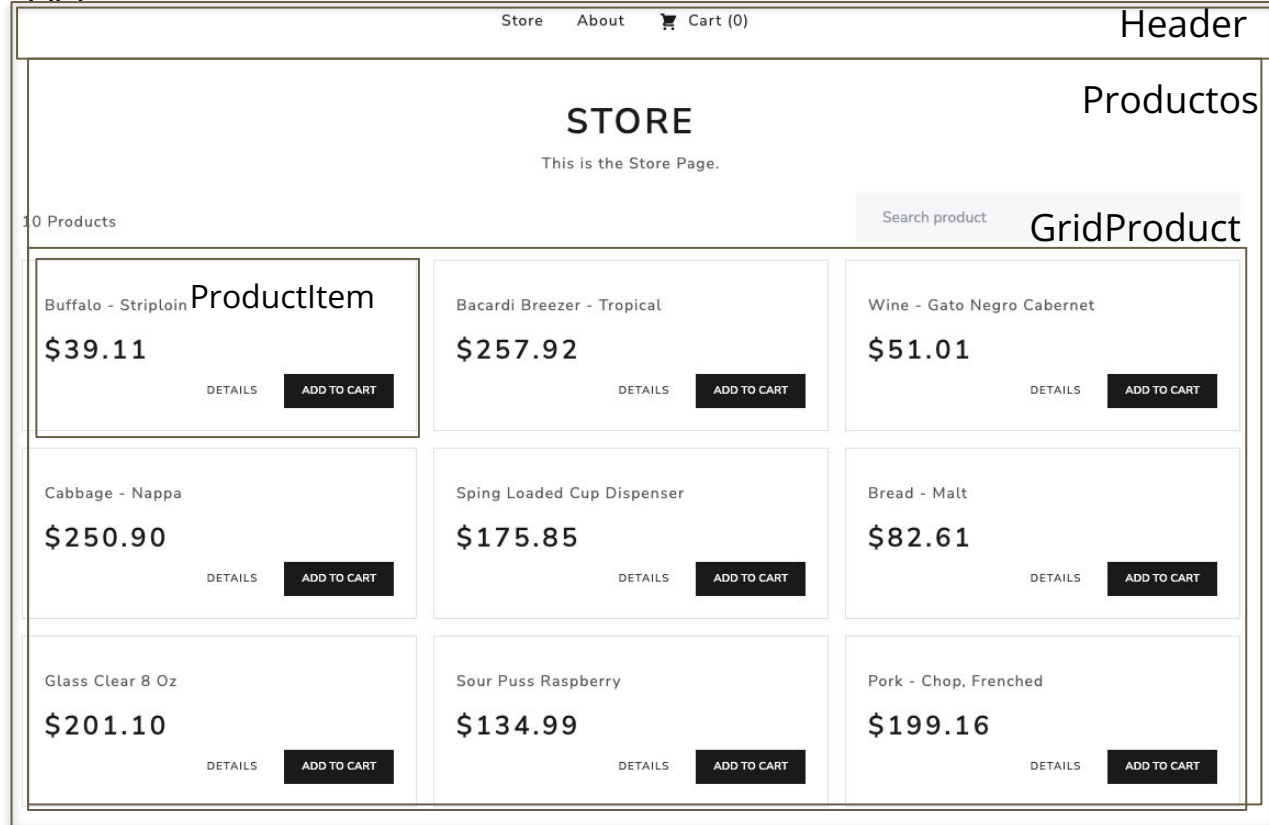
+Store

+About

+Cart

3. Implementar los componentes
4. Crear rutas

APP



Ejercicio en clases

Clinica de Mascota

LOGO

[Inicio](#) | [Listado propietarios](#) | [Agergar Mascotas](#)

Esta es la página de inicio....

orem ipsum dolor sit amet, consectetur adipiscing elit. Quisque consequat, odio ut sodales pretium, ligula dolor tempus nibh, accumsan scelerisque tellus nisl eu tellus. Aenean tempor lacus at nulla imperdiet, vitae sollicitudin mi tincidunt. Ut sit amet gravida sem. Suspendisse efficitur sapien eget pharetra lobortis. Etiam consectetur nec nisl sit amet aliquam. Proin risus enim, congue vitae aliquam sit amet, scelerisque eu eros. Suspendisse potenti. Duis suscipit, velit ac viverra congue, sem lacus aliquam felis, ac ultricies ipsum mi quis dui. Sed euismod, purus sed i



Listado de propietarios

# ID	Nombre	Dirección	Teléfono	# Opciones
1234	José Gomez Gomez	Calle 12 # 17 - 14 Los Robles	301111022	Ver/agregar mascotas
1234	José Gomez Gomez	Calle 12 # 17 - 14 Los Robles	301111022	Ver/agregar mascotas
1234	José Gomez Gomez	Calle 12 # 17 - 14 Los Robles	301111022	Ver/agregar mascotas
1234	José Gomez Gomez	Calle 12 # 17 - 14 Los Robles	301111022	Ver/agregar mascotas
1234	José Gomez Gomez	Calle 12 # 17 - 14 Los Robles	301111022	Ver/agregar mascotas

Agregar propietarios

Nombre

Dirección

Teléfono

+ Agregar

Esta es la página para agregar mascotas....

orem ipsum dolor sit amet, consectetur adipiscing elit. Quisque consequat, odio ut sodales pretium, ligula dolor tempus nibh, accumsan scelerisque tellus nisl eu tellus. Aenean tempor lacus at nulla imperdiet, vitae sollicitudin mi tincidunt. Ut sit amet gravida sem. Suspendisse efficitur sapien eget pharetra lobortis. Etiam consectetur nec nisl sit amet aliquam. Proin risus enim, congue vitae aliquam sit amet, scelerisque eu eros. Suspendisse potenti. Duis suscipit, velit ac viverra congue, sem lacus aliquam felis, ac ultricies ipsum mi quis dui. Sed euismod, purus sed i

Introducción a HOOKS

Un Hook es una función de javascript que permite crear/acceder al **estado** y a los **ciclos de vida** de React.

La incorporación de los hooks en React nos permite desarrollar **componentes funcionales con estado y ciclo de vida**

Reglas básicas para usar hooks:

- **Debe de ser llamado en el nivel superior de la aplicación - Un hook nunca debe de llamarse dentro de ciclos, condicionales o funciones anidadas**
- **Debe de llamarse en funciones o en otros hooks personalizados de React - Un hook nunca debe de ser llamado fuera de una función de React o de otro hook personalizado**

Hooks básicos

Hooks de estado: `useState`

Permite agregarle un estado local a un componente funcional y cambiar ese estado. Este hook nos devuelve un valor con un **estado inicial** y **una función** que es necesaria para actualizarlo.

Se crea una constante con un array con dos valores [**nombre**, **setNombre**] y luego asignarle un estado inicial con **useState**, **el parámetro de esta función recibe el valor del estado inicial**.

```
const [mivariableestado, setMiVariableestado] = useState(valorInicialMiVaribaleEstado)
```

Para actualizar el estado lo haremos con **setNombre**

```
<button onClick={() => setMiVariableestado(nuevoValor)}>Botón</button>
```

Hooks useState

```
class Name extends React.Component {  
  constructor(props) {  
    super(props);  
  
    this.state = {  
      name: props.name  
    };  
  }  
  
  changeName = e => {  
    this.setState({  
      name: e.target.value  
    });  
  };  
  
  render() {  
    return (  
      <div>  
        <input value={this.state.name} onChange={this.changeName} />  
        <h2>{this.state.name}</h2>  
      </div>  
    );  
  }  
}
```

```
function Name(props) {  
  const [name, changeName] = useState(props.name);  
  return (  
    <div>  
      <input value={name} onChange={e => changeName(e.target.value)} />  
      <h2>{name}</h2>  
    </div>  
  );  
}
```

Hooks básicos

Hook de efecto **useEffect**

Este hook nos permite agregar efectos secundarios a un componente funcional dado, es decir, nos permite realizar modificaciones en nuestra lógica una vez se haya realizado el renderizado, de la misma forma que los métodos de ciclo de vida **componentDidMount**, **componentDidUpdate** y **componentWillUnmount** en los componentes de clase.

Hooks básicos

Hook de efecto **useEffect**

Hay 3 formas distintas de invocar `useEffect`

Si no se especifica el segundo argumento `useEffect` tendrá el comportamiento por defecto, es decir, la arrow function será ejecutada cada vez que se complete un render del componente.

```
useEffect(() => { });
```

En caso de que especifiquemos un array como segundo argumento, la arrow function del primer argumento será ejecutada cada vez que se complete un render, pero solo si al menos uno de los valores que especifiquemos en el array cambia (es como un `componentDidMount` y un `componentDidUpdate` condicional).

```
useEffect(() => { }, [valor1, valor2, ...]);
```

si como segundo argumento de `useEffect` indicamos un array vacío `[]`, entonces la arrow function del primer argumento solo será ejecutada una vez, la primera vez que nuestro componente se renderice (es como un `componentDidMount`).

```
useEffect(() => { }, []);
```

Hooks básicos

```
class LoadUnload extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      data: null  
    }  
  }  
  
  componentDidMount() {  
    // función teórica que devuelve los datos de  
    // una cookie como objeto js  
    const loadedData = getCookie('data');  
    this.setState({data: loadedData})  
  }  
  
  componentWillUnmount() {  
    const { data } = this.state;  
    // Función teórica que guarda el objeto data en formato JSON  
    setCookie(data);  
  }  
}
```

```
function LoadUnload(props) {  
  const [data, setData] = useState(null);  
  
  useEffect(() => {  
    // similar a componentDidMount()  
  
    const loadedData = getCookie('data');  
    setData(loadedData);  
  
    // similar a componentWillUnmount()  
  
    return () => setCookie(data);  
  }, []);  
}
```

Hooks básicos

Hook de efecto useRef

Crear una “referencia” a un elemento del DOM creado durante el renderizado.

```
const refContainer = useRef(initialValue);
```

useRef devuelve un objeto ref mutable cuya propiedad **.current** se inicializa con el argumento pasado (initialValue). El objeto devuelto se mantendrá persistente durante la vida completa del componente.

```
function TextInputWithFocusButton() {  
  const inputEl = useRef(null);  
  const onClick = () => {  
    // `current` apunta al elemento de entrada de texto montado  
    inputEl.current.focus();  
  };  
  return (  
    <>  
      <input ref={inputEl} type="text" />  
      <button onClick={onClick}>Focus the input</button>  
    </>);  
}
```

Hook useReducer

El hook **useReducer** nos permite manejar estados “complejos” por medio de una función reductora.

Un estado complejo es aquel que tiene una estructura de varios niveles, por ejemplo: un objeto de objetos que contiene arrays.

```
{
  foo: {
    faa: {
      test: [],
      testB: ''
    }
  },
  fee: [],
  fii: {
    testC: [],
    testD: {}
  }
}
```

```
const initialState = {
  cartItems: [],
  itemCount: 0,
  total: 0.0,
  checkout: false
};
```

Hook useReducer

El hook **useReducer** tiene la siguiente estructura.

```
const [state, dispatch] = useReducer(  
  reducer,  
  initialArg,  
  init  
);
```

Donde **reducer** es simplemente una función de Javascript que actualiza el estado, **initialArg** es el estado inicial e **init** es una función opcional para también definir el estado inicial.

Este hook devuelve un array donde en la posición cero es el estado y en la posición uno es un dispatch que es una función que usaremos para actualizar el estado.

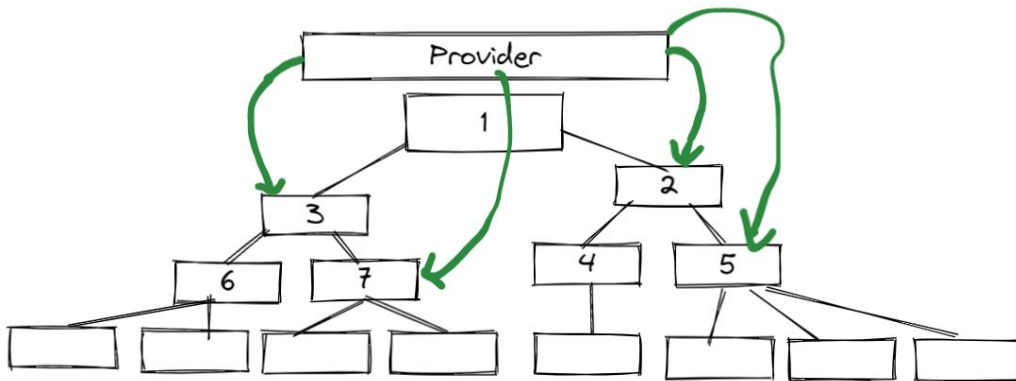
```
const [state, dispatch] = useReducer(CartReducer, initialState);
```

Hook useReducer

```
export const CartReducer = (state, action) => {
  switch (action.type) {
    case "ADD_ITEM":
      if (!state.cartItems.find((item) => item.id === action.payload.id)) {
      }
      let rtn = {
      };
      return rtn;
    case "REMOVE_ITEM":
      const newCartItems = state.cartItems.filter(...);
      return {
      };
    case "INCREASE":
      const newStateCartItems = {
      };
      const newState = {
      };
      return newState;
    case "DECREASE":
      const newcartItemsInDecrease = {
      };
      return {
      };
    case "CHECKOUT":
      return {
      };
    case "CLEAR": ...
    default:
      return state;
  }
};
```

Context API

La API de contexto puede pasar datos entre componentes a través del par Proveedor-Consumidor sin tener que pasar props. Usando la API de contexto cualquier componente puede acceder a los estados sin mucho esfuerzo. El proveedor puede actuar como un techo para que cualquier componente bajo ese techo pueda cosumirlo.



Context API

La API de contexto puede pasar datos entre componentes a través del par Proveedor-Consumidor sin tener que pasar props. Usando la API de contexto cualquier componente puede acceder a los estados sin mucho esfuerzo. El proveedor puede actuar como un techo para que cualquier componente bajo ese techo pueda cosumirlo.

Los 3 enfoques principales a la hora de añadir la API de contexto a su aplicación son los siguientes.

1. Crear un contexto usando `createContext`.
2. Envolver los componentes de React utilizando el contexto creado.
3. Consumir el contexto creado dentro de los componentes usando `useContext`.

Context API

```
export const CartContext = createContext();
const CartContextProvider = ({ children }) => {
  const [state, dispatch] = useReducer(CartReducer, initialState);
  const increase = (payload) => {
    dispatch({ type: "INCREASE", payload });
  };
  const decrease = (payload) => {
    dispatch({ type: "DECREASE", payload });
  };
  const addProduct = (payload) => {
    dispatch({ type: "ADD_ITEM", payload });
  };
  const removeProduct = (payload) => {
    dispatch({ type: "REMOVE_ITEM", payload });
  };
  const clearCart = () => {
    dispatch({ type: "CLEAR" });
  };
  const handleCheckout = () => {
    dispatch({ type: "CHECKOUT" });
  };
  const contextValues = {
    removeProduct, addProduct, increase, decrease, clearCart, handleCheckout, ...state
  };
  return (
    <CartContext.Provider value={contextValues}>
      {children}
    </CartContext.Provider>
  );
};
export default CartContextProvider;
```

Context API

Hacer que el contexto sea accesible por los componentes pertinentes.

Para hacer uso de los contextos creados tenemos que envolver nuestros componentes con los contextos. Vamos a hacerlo dentro del archivo **App.js**.

```
export default function App() {  
  return (  
    <CartContextProvider>  
      <Router>  
        <Header />  
        <div className="container">  
          <Routes />  
        </div>  
      </Router>  
    </CartContextProvider>  
  );  
}
```

Context API

Utilizar el contexto dentro de los componentes.

Para utilizar el contexto dentro del componente tenemos que importar el gancho **useContext**.

```
const CartItem = ({ product }) => {  
  const { increase, decrease, removeProduct } = useContext(CartContext);  
  
  return (  
    
```

Referencias

<https://es.reactjs.org/>

https://fullstackseries.com/post_series/aprende-react-js/

<https://ull-esit-sytw-1617.github.io/presentaciones-todos/react/>

<https://augdiaugus.gitbook.io/recoleccion-de-notas-publicas/escuela-de-javascript/curso-practico-de-react-js>

<https://ashwingoel.com/blog/a-comprehensive-guide-to-creating-your-first-react-app/>

<https://ed.team/blog/crea-rutas-con-react-router>

<https://blog.antarie.com/es/introduccion-react-hooks>